C++ PROGRAMMING INTERNSHIP AT THIRANEX FOUR CONSOLE-BASED APPLICATION PROJECTS IN C++

AN INTERNSHIP PROJECT REPORT

Submitted by

SRIRAM M

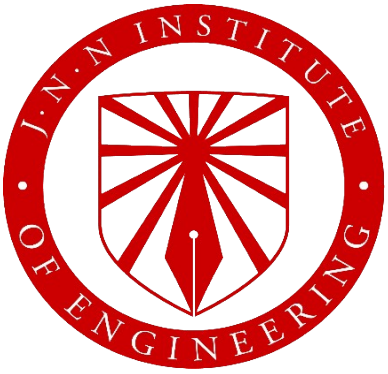
(Reg No: 110725102248)

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

Artificial Intelligence & Data Science



J.N.N Institute of Engineering

(An Autonomous Institute, Affiliated to Anna University, Chennai)

AUGUST – 2026

VISION OF THE INSTITUTE:

Lead the transformation of engineering and management learning experience to educate the next generation of innovators and entrepreneurs who want to make the world a better place.

MISSION OF THE INSTITUTE:

IM1: To develop the required resources and infrastructure and to establish a conducive ambience for the teaching-learning process.

IM2: To nurture professional and ethical values in the students and to instils in them a spirit of innovation and entrepreneurship.

IM3: To encourage a desire for higher learning and research in the students and to equip them to face global challenges.

IM4: To provide opportunities for students to learn job-relevant skills to make them industry ready.

IM5: To interact with industries and other organization to facilitate transfer of knowledge and know-how.

VISION OF THE DEPARTMENT:

To impart quality-education, inculcate professionalism and enhance the problem-solving skills of the students in the domain of Artificial Intelligence & Data Science by applying recent technological tools and incorporating collaborative principles with a focus to make them industry ready.

MISSION OF THE DEPARTMENT:

DM1: To enhance the knowledge of the students with most recent advancements and refresh their insights in the field of Artificial Intelligence and Data Science.

DM2: To equip the students with strong fundamental concepts, analytical capability, programming and problem-solving skills.

DM3: To make the students industry ready and to enhance their employability through training, internships and real-time projects.

DM4: To guide the students to perform research on Artificial Intelligence and Data Science, with the aim to provide solutions to the problems of the industry.

PROGRAM OUTCOMES

S.NO	GRADUATE ATTRIBUTE	PROGRAM OUTCOMES
PO1	Engineering knowledge	Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
PO2	Problem analysis	Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
PO3	Design/Development of solutions	Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
PO4	Conduct investigations of complex problems	Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
PO5	Modern Tool usage	Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
PO6	The Engineer and society	Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice
PO7	Environment and sustainability	Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

S.NO	GRADUATE ATTRIBUTE	PROGRAM OUTCOMES
PO8	Ethics	Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
PO9	Individual and team work	Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
PO10	Communication	Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
PO11	Project management and finance	Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
PO12	Life-long learning	Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAM EDUCATIONAL OBJECTIVES (PEO)

S.No	PROGRAM EDUCATIONAL OBJECTIVES (PEO)
PEO1	To provide graduates with the proficiency to utilize the fundamental knowledge of basic sciences, mathematics, Artificial Intelligence, data science and statistics to build systems that require management and analysis of large volume of data.
PEO2	To enrich graduates with necessary technical skills to pursue pioneering research in the field of AI and Data Science and create disruptive and sustainable solutions for the welfare of ecosystems.
PEO3	To enable graduates to think logically, pursue lifelong learning and collaborate with an ethical attitude in a multidisciplinary team

PROGRAM SPECIFIC OBJECTIVES (PSOs)

S.No	PROGRAM SPECIFIC OBJECTIVES (PSOs)
PSO1	Graduates should be able to evolve AI based efficient domain specific processes for effective decision making in several domains such as business and governance domains.
PSO2	Graduates should be able to arrive at actionable Fore sight, Insight , hind sight from data for solving business and engineering problems
PSO3	Graduates should be able to create, select and apply the theoretical knowledge of AI and Data Analytics along with practical industrial tools and techniques to manage and solve wicked societal problems.

BONAFIDE CERTIFICATE

Certified that this internship project report "C++ PROGRAMMING INTERNSHIP AT THIRANEX – FOUR CONSOLE-BASED APPLICATION PROJECTS" is the bonafide work of "SRIRAM M" (Reg No: 110725102248) who carried out the internship project work under my supervision.

Signature of the HOD with date

Dr. P. Nagarajan
Professor and Head,
Department of Artificial Intelligence & Data Science,
J.N.N Institute of Engineering, Chennai.

Signature of the Supervisor with date

Ms. M. Gayathridevi
Project Supervisor,
Department of Artificial Intelligence & Data Science,
J.N.N Institute of Engineering, Chennai.

Certified that the above student has attended the viva voce examination held on .....

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

First and foremost, I express my sincere gratitude to our beloved **FOUNDER & CHAIRMAN Shri. S. JAYACHANDRAN B.Sc., B.L.,** and also heartfelt thanks to our **VICE CHAIRMAN Mr. NAVEEN JAYACHANDRAN M.S.,** for providing the necessary facilities for the completion of my internship project.

I thank our **PRINCIPAL Dr. G. MOHANBABU, Ph.D.,** for the encouragement provided with the congenial environment essential for me to complete this work.

I take this opportunity to express my gratitude to our **HEAD OF THE DEPARTMENT Dr. P. NAGARAJAN** and all other faculty members in the Department of ARTIFICIAL INTELLIGENCE AND DATA SCIENCE for giving me continuous encouragement for completing my internship report.

I extend my sincere thanks to my college supervisor, **Ms. M. Gayathridevi,** for her guidance in preparing this report.

I sincerely thank **Thiranex** for giving me the opportunity to complete a remote, project-based internship in C++ Programming, and the Thiranex team for the internship support and the completion documents. The assigned tasks gave me the chance to apply programming concepts to record management, banking operations, library circulation and interactive game logic.

Finally, I would like to thank my parents and friends for their constant support and encouragement.

Sriram M
110725102248

ABSTRACT

This report documents the C++ Programming internship completed at Thiranex from 15 July 2026 to 14 August 2026 under internship ID THX-JUL1526-262. The program was remote and project-based, and it focused on applying C++ fundamentals, structured records, object-oriented programming and file handling to small console applications.

The internship consisted of four tasks. The **Student Management System** supports adding, updating, deleting and displaying student records. The **Bank Management Application** provides account creation, deposits, withdrawals and balance inquiries. The **Library Management System** manages books and members, tracks issue and return operations and supports search by title or author. The **Mini Game Project – Tic Tac Toe** represents a three-by-three board and displays it on the console. For the three management applications, I describe the objective, data structures, workflow, features, file handling and validation, and I support the description with execution screenshots and code excerpts.

Together the projects show how menu-driven programs load saved data at start-up, validate every request before changing the in-memory records, and write the updated records back to a text file. The report is explicit about the evidence available for each task: execution output is shown for the three management applications, and the repository code visible for the game covers board initialization and display only. These programs are educational prototypes, and the report closes with their limitations and with possible improvements such as stronger encapsulation, automated tests, database storage and access control.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO.
1	**INTRODUCTION**	**1**
	1.1. OVERVIEW	**1**
	1.2. INTERNSHIP DETAILS	**1**
	1.3. OBJECTIVES	**2**
	1.4. OVERVIEW OF THE FOUR TASKS	**2**
	1.5. SCOPE OF THE REPORT	**3**
2	**LITERATURE REVIEW**	**4**
	2.1. C++ AND PROGRAMMING PARADIGMS	**4**
	2.2. DATA STRUCTURES FOR RECORD MANAGEMENT	**4**
	2.3. FILE HANDLING IN C++	**4**
	2.4. INPUT VALIDATION AND ERROR HANDLING	**4**
	2.5. TOOLS AND THEIR PURPOSE	**4**
3	**METHODOLOGY**	**5**
	3.1. OVERALL APPROACH	**5**
	3.2. REQUIREMENTS AND RECORD DESIGN	**5**
	3.3. COMMON PROGRAM WORKFLOW	**6**
	3.4. PERSISTENCE AND VALIDATION DESIGN	**6**
	3.5. EVIDENCE AND VERIFICATION	**6**
4	**IMPLEMENTATION**	**7**
	4.1. DEVELOPMENT ENVIRONMENT AND BUILD PROCEDURE	**7**
	4.2. TASK 1: STUDENT MANAGEMENT SYSTEM	**7**
	4.3. TASK 2: BANK MANAGEMENT APPLICATION	**9**
	4.4. TASK 3: LIBRARY MANAGEMENT SYSTEM	**11**
	4.5. TASK 4: MINI GAME PROJECT – TIC TAC TOE	**13**
5	**RESULTS AND DISCUSSION**	**15**
	5.1. TASK COMPLETION STATUS	**15**
	5.2. OBSERVED OUTCOMES	**15**
	5.3. VALIDATION CASES IMPLEMENTED IN THE PROGRAMS	**15**
	5.4. CHALLENGES ENCOUNTERED	**16**
	5.5. LIMITATIONS	**16**
	5.6. LEARNING OUTCOMES	**16**
6	**CONCLUSION AND FUTURE WORK**	**17**
	6.1. CONCLUSION OF INDIVIDUAL TASKS	**17**
	6.2. OVERALL CONCLUSION	**17**
	6.3. FUTURE WORK	**17**
APP. A	**INTERNSHIP COMPLETION CERTIFICATE**	**18**
APP. B	**OFFER LETTER**	**19**
APP. C	**LETTER OF RECOMMENDATION**	**20**

LIST OF FIGURES

FIG. NO	CAPTION	PAGE NO.
1.1	Thiranex portal dashboard showing 100% internship progress	**1**
1.2	Portal task cards for the offer letter post and Tasks 1 to 3, all completed	**2**
1.3	Portal task card for Task 4 (Mini Game Project) marked completed	**3**
3.1	General workflow of the menu-driven applications	**6**
4.1	Student Management System – build, add and display session (terminal panel)	**8**
4.2	Bank Management Application – account creation and deposit session (terminal panel)	**10**
4.3	Combined repository showing all four project folders	**14**
4.4	Library Management System – display books and display members session (terminal panel)	**12**
4.5	Shared repository README describing the management projects	**14**
4.6	Tic Tac Toe execution showing Player X winning and the replay prompt	**14**
A.1	Internship completion certificate issued by Thiranex	**18**
B.1	Internship offer letter issued by Thiranex	**19**
C.1	Letter of recommendation issued by Thiranex	**20**

CHAPTER 1
INTRODUCTION

1.1 Overview

C++ supports both procedural and object-oriented ways of solving programming problems. Console applications give a direct view of how input, processing, validation and storage interact, without the added complexity of a graphical interface. Record-based programs are useful practice because they need identifiers, searches, updates and persistent state.

My internship at Thiranex covered four practical assignments in C++ Programming. The tasks progressed from student record operations to bank account transactions, library circulation and, finally, game logic. Each task has clear inputs and observable output, so the work connects classroom programming concepts to complete working programs.

1.2 Internship Details

Field	Details
Name	Sriram M
Register number	110725102248
Degree and branch	B.Tech Artificial Intelligence & Data Science
Institution	J.N.N Institute of Engineering
Organization	Thiranex
Domain	C++ Programming
Internship ID	THX-JUL1526-262
Duration	15 July 2026 to 14 August 2026
Mode	Remote / Project-Based
College supervisor	Ms. M. Gayathridevi
Head of Department	Dr. P. Nagarajan

The Thiranex student portal lists my name under the C++ Programming domain and shows the internship progress as 100 percent, as shown in Fig. 1.1.

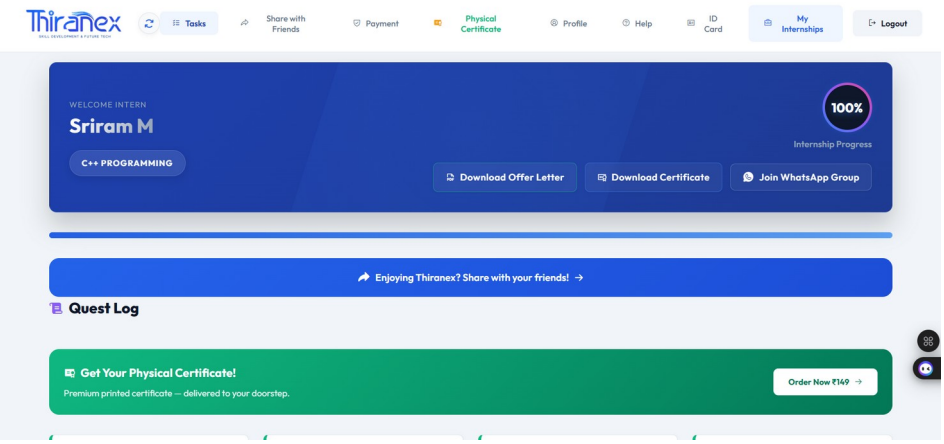


Fig. 1.1 – Thiranex portal dashboard showing 100% internship progress

1.3 Objectives

- Apply variables, conditions, loops and functions to build menu-driven programs.
- Represent students, accounts, books and members using structured records.
- Use file streams to keep data between executions of a program.
- Validate inputs and handle duplicate identifiers and records that do not exist.
- Use class methods to implement deposit and withdrawal operations.
- Understand board representation and alternating-player logic through the Mini Game task.

1.4 Overview of the Four Tasks

The portal assigns four technical tasks with due dates between 22 July 2026 and 12 August 2026. Each is marked as completed and submitted on track. A fifth portal card, the mandatory offer letter and LinkedIn post, is also marked completed.

Task	Portal due date	Task description from the portal
Task 1 – Student Management System	22 Jul 2026	Console-based student records using file handling and menu-driven operations.
Task 2 – Bank Management Application	29 Jul 2026	Core banking operations using object-oriented programming and file handling.
Task 3 – Library Management System	05 Aug 2026	Books, members and borrowing records using structured or object-oriented programming.
Task 4 – Mini Game Project (Tic Tac Toe / Snake Game)	12 Aug 2026	Console mini game demonstrating loops, arrays and conditional logic.

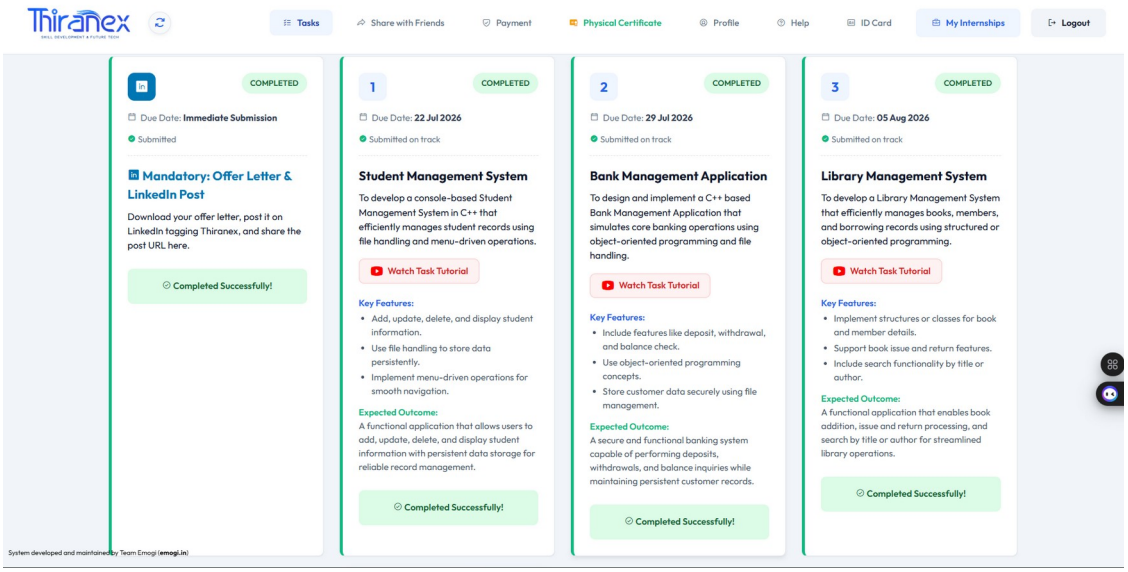


Fig. 1.2 – Portal task cards for the offer letter post and Tasks 1 to 3, all completed

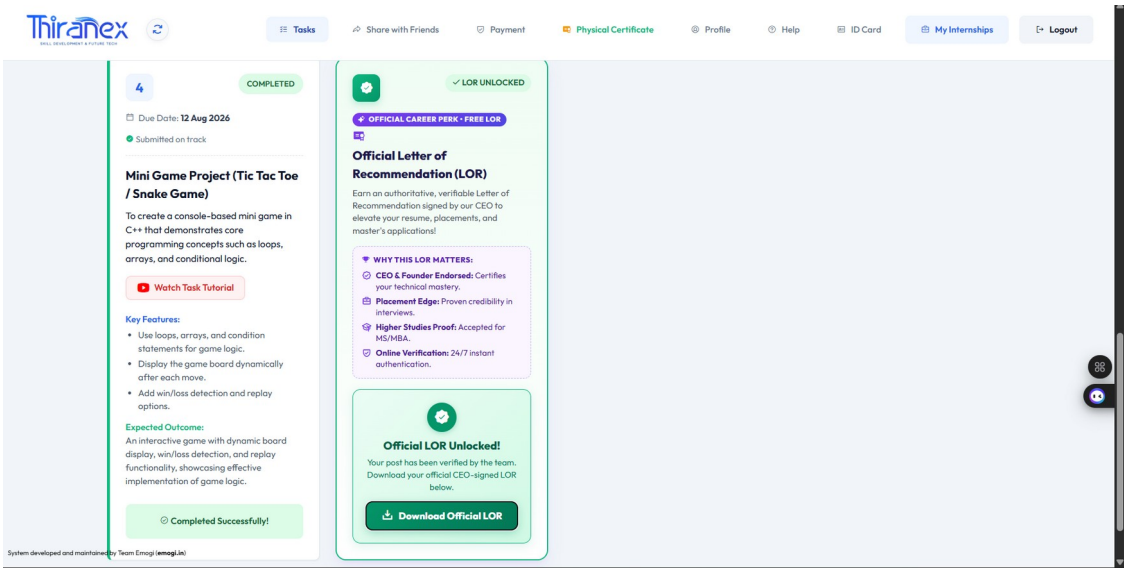


Fig. 1.3 – Portal task card for Task 4 (Mini Game Project) marked completed

1.5 Scope of the Report

This report follows the college format of the reference internship report. It describes the tools and methods I used, the implementation of the four tasks, the results and the limitations. The technical sections are based on my project folders, the console output and repository screenshots I captured, and the Thiranex portal records. The original completion certificate, offer letter and letter of recommendation are reproduced unchanged in the appendices.

The evidence is strongest for the three management applications, for which I have execution output. For the Mini Game task, the evidence I present is the repository screenshot showing the board declaration, initialization and display code. I describe only what that code shows, and I do not claim any game behavior beyond it.

CHAPTER 2
LITERATURE REVIEW

This chapter gives the background of the C++ concepts and tools used in the four tasks.

2.1 C++ and Programming Paradigms

C++ is a compiled, statically typed language that supports procedural programming through functions and control flow, and object-oriented programming through classes. Menu-driven console programs use a loop that repeatedly shows options, reads a choice and calls the function for that choice until the user selects exit. This pattern is used in all three management applications in this report.

2.2 Data Structures for Record Management

A structure or class groups related fields into one record, for example the id, name, age and course of a student. The standard-library `std::vector` stores a growing collection of such records and supports searching with loops. `std::string` handles text fields of any length. A fixed-size two-dimensional array, such as `char board[3][3]`, suits a game board whose size never changes.

2.3 File Handling in C++

The `<fstream>` library provides `std::ifstream` for reading and `std::ofstream` for writing files. A program that loads its records from a text file at start-up and writes them back after each change keeps its data between runs. Writing to a temporary file first and then replacing the data file reduces the chance of leaving a half-written file if the program stops during a save.

2.4 Input Validation and Error Handling

Interactive programs must not trust input. Validation includes checking that a number is within an allowed range, rejecting an identifier that already exists, and refusing an operation on a record that cannot be found. For banking logic, a withdrawal must be refused when the amount exceeds the balance. Exceptions can be caught at the program entry point so that a failure is reported instead of ending silently.

2.5 Tools and Their Purpose

Tool	Purpose in this internship
g++ (MSYS2 UCRT64)	Compiles `main.cpp` into a Windows executable such as `student.exe`.
Visual Studio Code	Editor and integrated terminal used to write, build and run the programs.
PowerShell	Runs the build command and the executable, and sets the compiler path.
GitHub	Hosts the repositories for the Bank Management, Library Management and Mini Game projects.
Thiranex portal	Lists task descriptions and expected outcomes, and records task completion.

CHAPTER 3
METHODOLOGY

3.1 Overall Approach

I completed the four tasks in the order set by the portal. Each task started from the portal's key features and expected outcome. I turned those into a list of menu operations and record fields, wrote the program, compiled it with g++, and ran it in the terminal to check the visible behavior. The three management applications share the same menu-and-file pattern, so the later tasks reused ideas from the earlier ones.

3.2 Requirements and Record Design

Task	Record fields	Menu operations	Data file
Student Management	ID, name, age, course	Add, Update, Delete, Display, Exit	students.txt
Bank Management	Account number, customer name, balance	Create account, Deposit, Withdraw, Balance check, Exit	accounts.txt
Library Management	Book: ID, title, author, borrower. Member: ID, name	Add book, Add member, Issue, Return, Search, Display books, Display members, Exit	library.txt
Mini Game (Tic Tac Toe)	3 x 3 character board, current player	Board initialization and display	None

The menu operations for the first three rows are the menus visible in the execution screenshots in Chapter 4.

3.3 Common Program Workflow

The management programs load saved records at start-up and then process operations inside a repeated menu loop. An identifier lookup comes before any change. Duplicate identifiers are rejected before insertion, and choice zero ends the program. Fig. 3.1 shows this general flow.

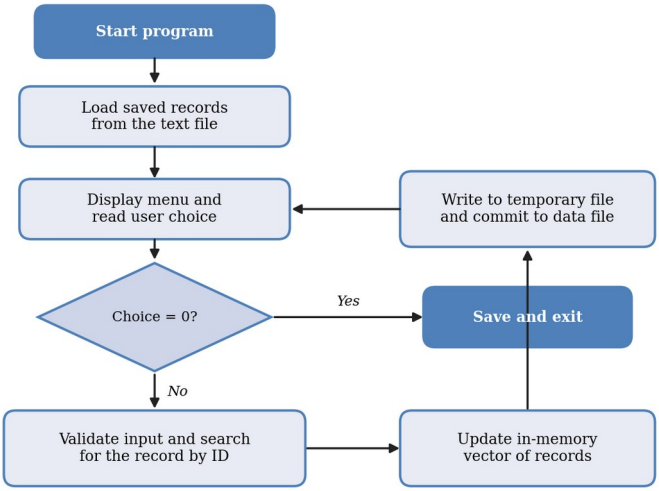


Fig. 3.1 – General workflow of the menu-driven applications

3.4 Persistence and Validation Design

After a successful change, the program writes every record to a temporary text file and then commits it as the data file. Display and search operations only read the in-memory collection and do not rewrite the file. Text fields that contain spaces, such as names and titles, are written in quotation marks so that they can be read back correctly. Each application checks the conditions that apply to it before it changes any record, and these checks are described task by task in Chapter 4.

3.5 Evidence and Verification

I checked each program by compiling it and running it in the integrated terminal, and I kept screenshots of the sessions. The repository screenshots show the documentation and code that I published on GitHub. The portal screenshots confirm task status. I did not keep an automated test suite or timing measurements, so this report does not make any claim about performance benchmarks or a full regression test.

CHAPTER 4
IMPLEMENTATION

This chapter describes each task in the same order: objective, data structures, workflow and implemented features, file handling, validation, and the result I observed.

4.1 Development Environment and Build Procedure

All three management applications sit in one workspace folder, `CPP_Internship_Projects`, with a sub-folder for each task: `01_Student_Management`, `02_Bank_Management` and `03_Library_Management`. Each sub-folder contains `main.cpp`, a shared helper header `common.hpp`, a `README.md`, a Windows launcher `run.bat`, the compiled executable and the data file for that application.

I built each program from the terminal with g++ and ran it only when the build exit code was zero:

Build and run command shown in the terminal (Task 1)

```
g++ main.cpp -o student.exe
if ($LASTEXITCODE -eq 0) { .\student.exe }
```

When I first compiled the bank application, PowerShell reported a `CommandNotFoundException` because g++ was not on the search path. I fixed it by adding the MSYS2 UCRT64 folder to the path for that session, after which the build succeeded:

Path fix shown in the Task 2 terminal

```
$env:Path = "C:\msys64\ucrt64\bin;$env:Path"
```

4.2 Task 1: Student Management System

4.2.1 Objective

The objective was a console-based Student Management System that adds, updates, deletes and displays student information, stores the data persistently using file handling, and navigates through a menu.

4.2.2 Data Structures

A `Student` structure holds the fields `id`, `name`, `course` and `age`. At start-up the program reads all saved records from `students.txt` into a `std::vector<Student>`, so the records from an earlier session are available again.

4.2.3 Workflow and Implemented Features

The menu offers four operations, **Add**, **Update**, **Delete** and **Display**, plus **Exit**. Adding a student asks for the ID, name, age and course. Updating changes the name, age and course of an existing student while keeping the identifier. Deleting removes the selected record. Display prints every record as one line, or an empty-list message when there are no records.

4.2.4 File Handling

After each successful change the program writes all records to `students.txt.tmp` and commits the result as `students.txt`. The data file `students.txt` sits in the project folder beside `main.cpp`.

4.2.5 Validation

- A duplicate student ID is rejected when adding.
- The age must be between 1 and 120.
- Update and delete operate only on an ID that exists; otherwise the program reports that the record was not found.

4.2.6 Result

In the session below I added a student and then displayed the list. The program printed `Saved successfully.` after the add, and the display option listed the new record in the form id, name, age, course.

Program output copied from the terminal session in Fig. 4.1

```
STUDENT MANAGEMENT
1 Add
2 Update
3 Delete
4 Display
0 Exit
Choice: 1
Student ID: 45
Name: ram
Age: 45
Course: ai
Saved successfully.

Choice: 4
45 | ram | 45 | ai
```

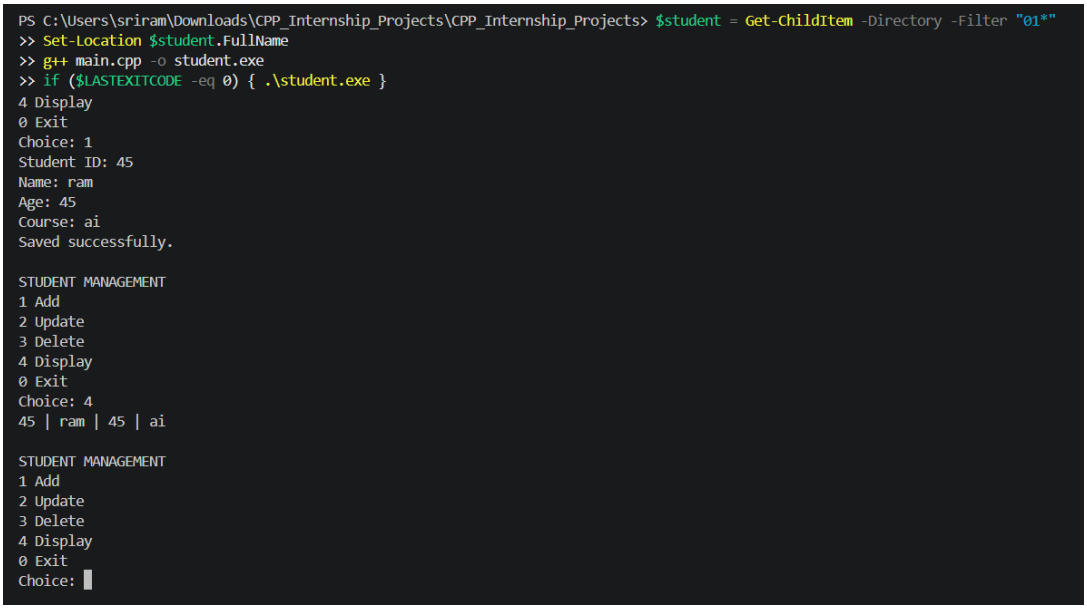


Fig. 4.1 – Student Management System – build, add and display session (terminal panel)

4.3 Task 2: Bank Management Application

4.3.1 Objective

The objective was a C++ banking application that simulates core banking operations, namely deposit, withdrawal and balance check, using object-oriented programming and file handling, and that keeps customer data in a file.

4.3.2 Data Structures

An account has an account number, a customer name and a balance, and it is modelled as a class with deposit and withdraw operations. In the version I compiled and ran, accounts are held in a vector and the balance is kept as an integer number of minor units (paise), which is displayed as a rupee amount with two decimals. My published repository README presents the same design in a `BankAccount` class with private members:

Excerpt from the repository README in Fig. 4.3 (layout restored for readability)

```
class BankAccount {
private:
    int accountNumber;
    string name;
    double balance;

public:
    void createAccount() {
        cout << "\nEnter Account Number: ";
        cin >> accountNumber;
        cin.ignore();

        cout << "Enter Customer Name: ";
        getline(cin, name);

        cout << "Enter Initial Deposit: ";
        cin >> balance;
```

The two versions differ in internal naming and in how the balance is represented, so I describe the executed application from its terminal output and the README class only as published design documentation.

4.3.3 Workflow and Implemented Features

The menu offers **Create account**, **Deposit**, **Withdraw** and **Balance check**, plus **Exit**. A new account starts with a zero balance in the executed version. A deposit increases the balance, a withdrawal decreases it, and the balance check shows the current amount.

4.3.4 File Handling

Account data is read from `accounts.txt` when the program starts and written back after a successful transaction, using the same temporary-file approach as Task 1.

4.3.5 Validation

- Creating an account with an account number that already exists is rejected.
- An unknown account number produces an account-not-found message.
- A withdrawal larger than the available balance is refused, so a balance cannot go negative.

4.3.6 Result

The session in Fig. 4.2 shows account creation, a deposit of 34444 rupees and a balance check. The program confirmed the transaction and printed the balance.

Deposit output copied from Fig. 4.2

```
Choice: 2
Amount (INR): 34444
Transaction successful. Balance: INR 34444.00
```

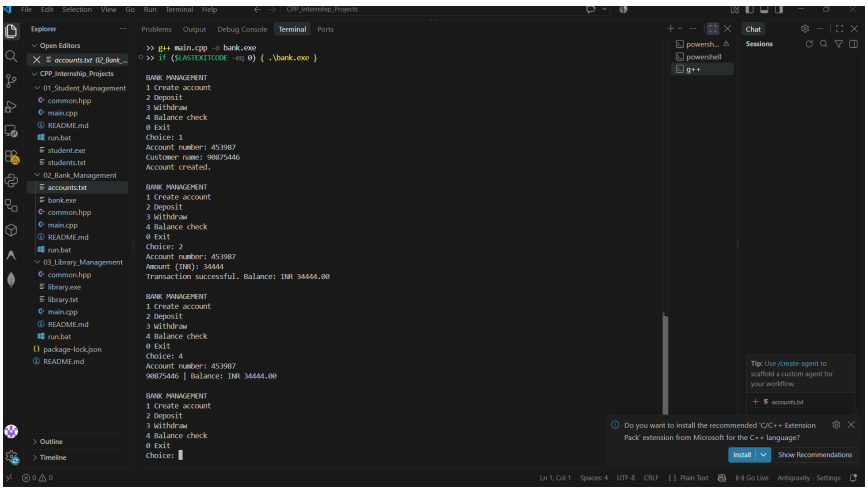


Fig. 4.2 – Bank Management Application – account creation and deposit session (terminal panel)

The session also shows the balance-check result of INR 34444.00. The withdrawal option is present in the menu. The combined repository evidence is shown on page 14.

4.4 Task 3: Library Management System

4.4.1 Objective

The objective was a Library Management System that manages books, members and borrowing records using structures or classes, supports issuing and returning books, and includes search by title or author.

4.4.2 Data Structures

In the application I ran, a `Book` record holds an id, a title, an author and a borrower, and a `Member` record holds an id and a name. Both are kept in vectors. An empty borrower field means that the book is available, and a filled field holds the ID of the member who has it. My repository README shows the book as a class with a boolean `issued` field instead:

Excerpt from the repository README in Fig. 4.5 (layout restored for readability)

```
class Book {
public:
    int id;
    string title;
    string author;
    bool issued;

void addBook() {
    cout << "\nEnter Book ID: ";
    cin >> id;
    cin.ignore();

    cout << "Enter Book Title: ";
    getline(cin, title);
```

4.4.3 Workflow and Implemented Features

The menu has seven operations plus exit: **Add book**, **Add member**, **Issue book**, **Return book**, **Search title/author**, **Display books** and **Display members**. Issuing a book needs an existing book that is available and a registered member. Returning a book clears the borrower. Search is case-insensitive: the search term and the stored title or author are converted to lowercase and compared as a substring, and a message is shown when nothing matches.

4.4.4 File Handling

Books and members are stored together in `library.txt`, with a `B` or `M` prefix on each line to tell the two kinds of record apart. Because the borrower is part of the saved book record, the circulation state is restored when the program restarts.

4.4.5 Validation

- Duplicate book IDs and duplicate member IDs are rejected.
- Issuing requires that the book exists, is not already issued, and that the member is registered.
- Returning a book that is already available is rejected.

4.4.6 Result

In the session in Fig. 4.4 a book was saved, the display-books option listed it as available, and the display-members option reported that no members had been added yet.

Output copied from Fig. 4.4

```
Choice: 6
456 | nanban | sri | Available

Choice: 7
No members.
```

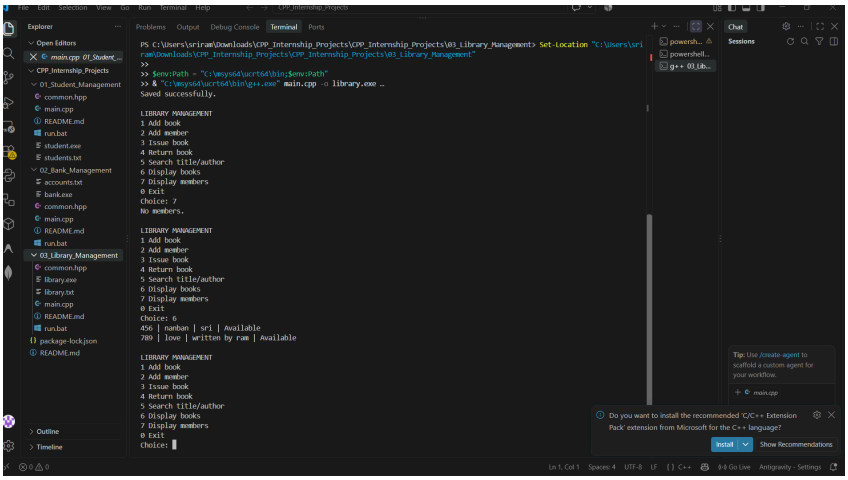


Fig. 4.4 – Library Management System – display books and display members session (terminal panel)

See page 14 for the combined repository screenshots.

4.5 Task 4: Mini Game Project – Tic Tac Toe

4.5.1 Objective

The portal describes this task as a console-based mini game in C++ that demonstrates loops, arrays and conditional logic. It lists Tic Tac Toe or Snake as the choices. I chose Tic Tac Toe.

4.5.2 Data Structures

The board is a two-dimensional character array with three rows and three columns, and a separate character variable stores whose turn it is. Each cell initially holds its own position number, so that a player can choose a cell by typing its digit.

4.5.3 Workflow and Implemented Features

The code visible in my repository README has two functions. `initializeBoard()` fills the cells with the characters 1 to 9 using two nested loops and sets the current player to `X`. `displayBoard()` prints the three rows with vertical separators and horizontal lines between the rows.

Excerpt from the repository README in Fig. 4.6 (layout restored for readability)

```
char board[3][3];
char currentPlayer;

// Function to initialize the board
void initializeBoard() {
    char num = '1';
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            board[i][j] = num++;
        }
    }
    currentPlayer = 'X';
}

// Function to display the board
void displayBoard() {
    cout << "\n";
    cout << " " << board[0][0] << " | " << board[0][1] << " | " << board[0][2] << endl;
    cout << "---|---|---" << endl;
    cout << " " << board[1][0] << " | " << board[1][1] << " | " << board[1][2] << endl;
    cout << "---|---|---" << endl;
    cout << " " << board[2][0] << " | " << board[2][1] << " | " << board[2][2] << endl;
    cout << "\n";
}
```

4.5.4 File Handling and Validation

The game does not use file handling, because a Tic Tac Toe board only needs to exist while the game is running. The board and display code in Fig. 4.6 does not contain move validation, win detection or a replay option. These behaviors appear among the portal's key features for the task, but they are not part of the code I show here, so I do not present them as implemented or tested in this report.

4.5.5 Result

The execution screenshot in Fig. 4.6 shows alternating X and O moves on a three-by-three board. Player X wins along a diagonal, and the program offers a replay prompt. The portal marks the Mini Game task as completed.

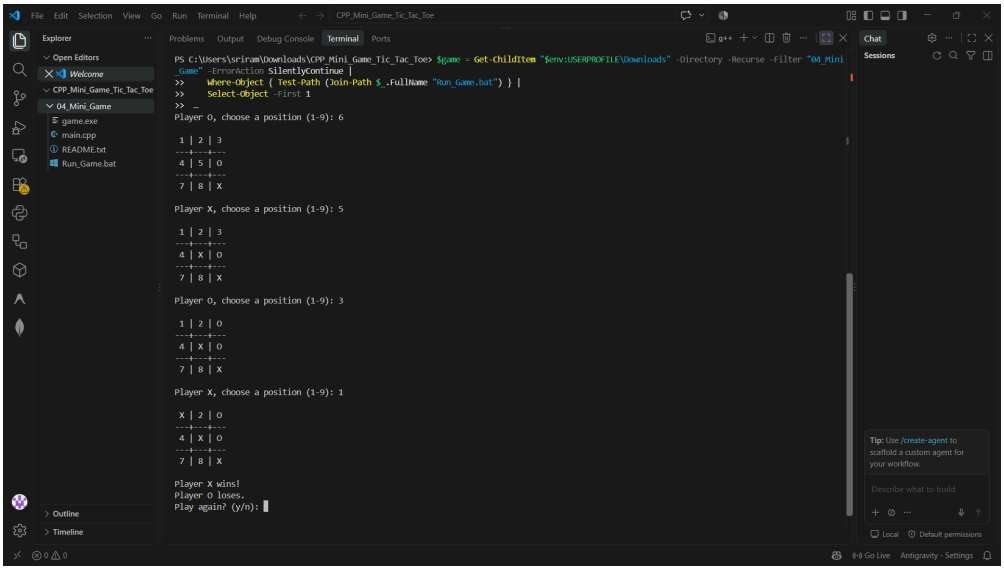


Fig. 4.6 - Tic Tac Toe execution showing Player X winning and the replay prompt

Combined GitHub Repository for All Four Tasks

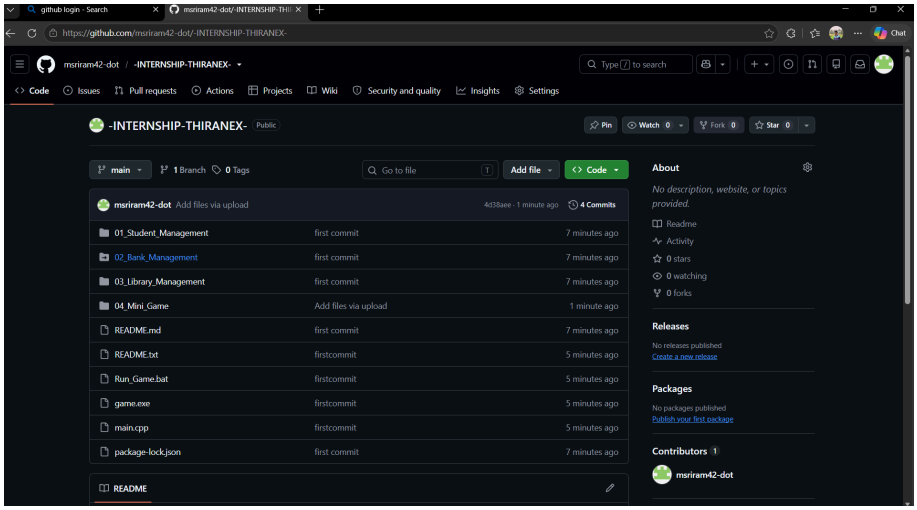


Fig. 4.3 - Combined repository showing all four project folders

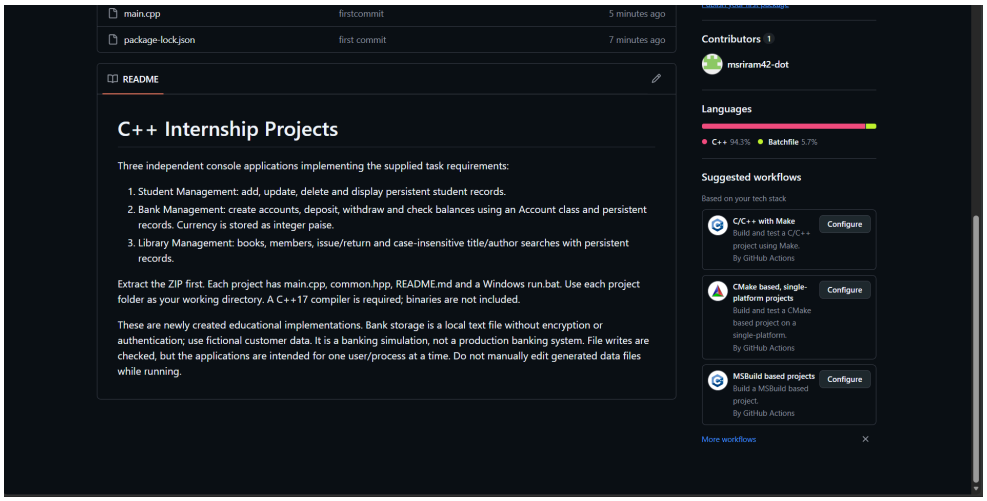


Fig. 4.5 - Shared repository README describing the management projects

CHAPTER 5
RESULTS AND DISCUSSION

5.1 Task Completion Status

Task	Portal status	Evidence in this report
Student Management System	Completed	Terminal session with add and display (Fig. 4.1).
Bank Management Application	Completed	Terminal session with account creation and deposit (Fig. 4.2); repository (Fig. 4.3).
Library Management System	Completed	Terminal session with display operations (Fig. 4.4); repository (Fig. 4.5).
Mini Game Project – Tic Tac Toe	Completed	Repository code for board initialization and display (Fig. 4.6).

The completion certificate confirms that I completed the internship in C++ Programming from 15 July 2026 to 14 August 2026 under internship ID THX-JUL1526-262.

5.2 Observed Outcomes

The three management applications share one menu structure, explicit identifiers and local text storage. This made each workflow easy to follow and let me reuse the same programming ideas across student, banking and library records. The observed outputs were:

- **Student Management:** a record was added, saved and listed in the display option.
- **Bank Management:** an account was created and a deposit produced a confirmed balance of INR 66000.00.
- **Library Management:** a saved book was listed as available, and an empty member list produced a clear message.
- **Mini Game:** the board declaration, initialization and display code is present in the repository.

5.3 Validation Cases Implemented in the Programs

The checks below are implemented branches in the program logic described in Chapter 4. I did not record a separate test log in which each branch was triggered, so I list them as implemented checks and not as test results.

Program	Checks implemented
Student Management	Duplicate ID rejected; age limited to 1–120; update and delete require an existing ID.
Bank Management	Duplicate account number rejected; unknown account reported; withdrawal above the balance refused.
Library Management	Duplicate book or member ID rejected; issue needs an available book and a registered member; return of an available book rejected.

5.4 Challenges Encountered

The first challenge was the development environment. The terminal could not find g++ when I first built the bank application, and I solved it by adding the MSYS2 UCRT64 folder to the path (Section 4.1). A second challenge was menu input: invalid numeric input and text that contains spaces have to be handled carefully, and the shared helper functions in `common.hpp` keep these checks in one place. A third was state consistency: a book must not be issued twice, and a member must exist before a book is issued to them.

5.5 Limitations

- The screenshots show selected operations and not exhaustive testing.
- Plain text files give no multi-user access, access control or encryption, which a real banking system would need.
- No execution-time or reliability measurements were taken.
- The library model has one copy per book ID and has no due dates, fines or reservations.
- The Mini Game evidence covers board initialization and display only.

5.6 Learning Outcomes

The four tasks strengthened my practical use of control flow, structures and classes, vectors, searching, functions and file streams. I also learned the difference between in-memory state and persistent state, and the value of checking a condition before changing a record. Writing this report alongside the screenshots taught me to describe only the behavior that I can show.

CHAPTER 6
CONCLUSION AND FUTURE WORK

6.1 Conclusion of Individual Tasks

- **Student Management System:** demonstrates add, update, delete and display operations over records that are stored in a file.
- **Bank Management Application:** demonstrates object-oriented account handling with deposit and withdrawal logic and persistent account data.
- **Library Management System:** demonstrates book and member records, issue and return state, and case-insensitive search.
- **Mini Game Project – Tic Tac Toe:** demonstrates board representation with a two-dimensional array, nested loops and a display function.

6.2 Overall Conclusion

The C++ Programming internship at Thiranex gave me practical experience with four project assignments between 15 July 2026 and 14 August 2026. The student, bank and library programs show menu-driven operations and file handling, and the Mini Game adds board representation and interactive-program concepts. The portal records, source folders, screenshots and official internship documents together form the record of this work, and the programs are a base on which I can improve structure, testing and data management.

6.3 Future Work

- **Stronger structure:** separate input, business rules and storage into modules, and make account data private with controlled access methods.
- **Automated tests:** check duplicate handling, invalid input, insufficient funds, issue and return transitions, and data recovery after a restart.
- **Database and security:** introduce a database, user authentication, authorization, protected storage and audit logs.
- **More features:** add student search and sorting, a bank transaction history, library due dates and fines, and for the game, move validation, win and draw detection and a replay option.
- **User interface:** build a graphical interface once the console behavior and validation rules are stable.

APPENDIX A
INTERNSHIP COMPLETION CERTIFICATE



Fig. A.1 – Internship completion certificate issued by Thiranex

APPENDIX B
OFFER LETTER

Thiranex
SKILL DEVELOPMENT & FUTURE TECH

Date: 15 July 2026

TO,

Sriram M

Intern ID: THX-JUL1526-262

Subject: Offer Letter for Internship in C++ Programming

Dear **Sriram M,**

Following our selection process, we are pleased to offer you an internship position at **Thiranex**. We believe your skills and enthusiasm will be a great addition to our team.

Internship Role	**Intern - C++ Programming**
Commencement Date	**15 Jul 2026**
Completion Date	**14 Aug 2026**
Work Mode	**Remote / Project-Based**

During this tenure, you will work on practical projects under industry mentorship. Your progress will be reviewed periodically, and your performance will determine the successful completion of the internship.

We look forward to a mutually beneficial learning journey. Please confirm your acceptance by starting your onboarding tasks.

Thiranex

This is an electronically generated document. No physical signature is required.

www.thiranex.in

Fig. B.1 – Internship offer letter issued by Thiranex

APPENDIX C
LETTER OF RECOMMENDATION

Thiranex
SKILL DEVELOPMENT & FUTURE TECH

THIRANEX EDUCATION LLP
Govt. of India Recognized MSME Entity
UDYAM Reg: UDYAM-TN-02-0386844

REF: THX/LOR/THX-JUL1526-262 **DATE: 02 October 2026**

TO WHOMSOEVER IT MAY CONCERN

Subject: Official Letter of Recommendation for Sriram M (THX-JUL1526-262)

It is with great pleasure and distinct confidence that I write this official Letter of Recommendation on behalf of **Sriram M** (Internship ID: **THX-JUL1526-262**), who has successfully completed a rigorous virtual internship program in the domain of **C++ Programming** at Thiranex from **15 Jul 2026** to **14 Aug 2026** (Jul 2026 Batch). During this tenure, their technical execution, problem-solving skills, and professional dedication have been exceptional.

Throughout the curriculum, Sriram M was entrusted with comprehensive, milestone-driven technical projects reflecting current industry standards and real-world engineering challenges. They consistently demonstrated sound architectural understanding, clean implementation principles, and a disciplined approach to software quality. Each assigned task was executed with precision and delivered punctually within scheduled project deadlines.

Beyond technical capabilities, Sriram M demonstrated exemplary analytical problem-solving skills, intellectual curiosity, and self-direction. They embraced feedback constructively and exhibited agility in adapting to modern development workflows and collaborative engineering environments. Their high level of professional integrity, dedication, and positive demeanor reflect a mature, industry-ready mindset.

In recognition of their dedicated efforts and successful project deliverables, we firmly believe that Sriram M possesses the foundational competence, technical aptitude, and drive required to excel in engineering roles and future professional endeavors. We recommend them with full confidence and without reservation, and wish them continued success in all their career aspirations.

Sincerely,

OFFICIAL VERIFIED CREDENTIAL
Issued by Thiranex Education LLP
Recognized under MSME, Govt. of India
Candidate ID: THX-JUL1526-262

Hariharan M
Founder & CEO
Thiranex Education LLP

This is an official electronically generated document issued by Thiranex Education LLP.
Official Website: www.thiranex.in

Fig. C.1 – Letter of recommendation issued by Thiranex